

kelompok D

PROJECT ALPRO

DOSEN PENGAMPU:

Lutfiah Maharani Siniwi, S.Stat., M.Stat.

Anggota Kelompok:

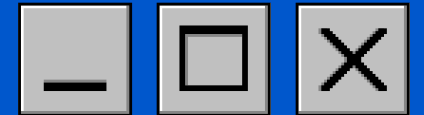
Meyluna Prachi Sri Rumondang Purba (K1D025091)

Khalisa Irfani (K1D025093)

Nadien Dwi.A. (K1D025098)

Safira Aulia (K1D025103)





LATAR BELAKANG

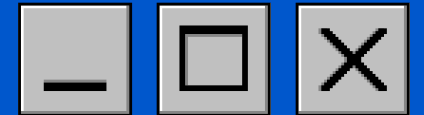
Algoritma dan pemrograman merupakan dasar dalam pengembangan perangkat lunak dan penyelesaian masalah komputasional. Melalui project ini, mahasiswa menerapkan konsep algoritma, pseudocode, flowchart, serta implementasi program menggunakan bahasa Python dan R pada berbagai studi kasus, seperti pengolahan teks, perhitungan matematis, pengolahan data, klasifikasi cluster, dan statistika.

TUJUAN

- Menganalisis permasalahan komputasional berdasarkan konsep input, proses, dan output.
- Menyusun algoritma dalam bentuk pseudocode dan flowchart.
- Mengimplementasikan program menggunakan Python dan R.
- Melakukan pengujian program serta menangani kondisi khusus dan validasi input.
- Membandingkan logika pemrograman pada Python dan R dalam menyelesaikan masalah yang sama.



PROGRAM 1



DESKRIPSI SOAL

Program dibuat untuk menghitung seluruh jumlah kata dan jumlah kalimat dalam sebuah teks. Pengguna memasukkan sebuah teks yang terdiri dari satu atau lebih kalimat. Program kemudian menghitung banyak kata dan banyak kalimat berdasarkan tanda titik (.) sebagai penanda akhir kalimat.

KOMPONEN	KETERANGAN
INPUT	Sebuah teks dari pengguna
PROSES	Menghitung jumlah kata menggunakan pemisahan berdasarkan spasi dan menghitung jumlah kalimat berdasarkan jumlah tanda titik
OUTPUT	Jumlah kata dan jumlah kalimat

BATASAN MASALAH

- Tanda titik (.) hanya digunakan untuk mengakhiri kalimat.
- Tidak ada singkatan seperti "S.H.", "Dr.", "dll."
- Kata dipisahkan oleh spasi.
- Input berupa teks biasa

KONDISI KHUSUS

1. Input teks kosong
Program menghasilkan jumlah kata = 0 dan jumlah kalimat = 0.
2. Teks tanpa tanda titik (.)
Jumlah kata tetap dihitung, sedangkan jumlah kalimat = 0.
3. Teks terdiri dari beberapa kalimat
Setiap tanda titik (.) dihitung sebagai akhir satu kalimat sesuai asumsi soal.

SOURCE KODE PYTHON

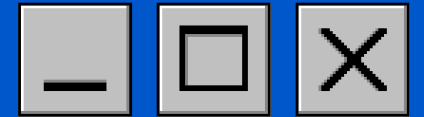
```
# Program Menghitung Jumlah Kata dan Jumlah Kalimat
```

```
teks = input("Masukkan teks: ")
```

```
if teks.strip() == "":  
    jumlah_kata = 0  
    jumlah_kalimat = 0  
else:  
    jumlah_kata = len(teks.split())  
    jumlah_kalimat = len(teks.split("."))-1
```

```
print("\nTeks tersebut memuat", jumlah_kalimat,  
      "kalimat dan", jumlah_kata, "kata.")
```

PROGRAM 1



SOURCE KODE R

```
{
  # Membaca input teks dari pengguna
  cat("Masukkan teks: ")
  teks <- readline()

  # Validasi jika input kosong (setelah menghapus spasi di ujung)
  teks_trim <- trimws(teks)

  if (teks_trim == "") {
    jumlah_kata <- 0
    jumlah_kalimat <- 0
  } else {
    # Menghitung jumlah kata dengan memisahkan berdasarkan spasi
    # menggunakan strsplit
    kata_list <- strsplit(teks_trim, "\\s+")[1]
    jumlah_kata <- length(kata_list)

    # Menghitung jumlah kalimat dengan menghitung frekuensi tanda titik
    # (.)
    pencocokan_titik <- gregexpr("\\.", teks)[1]

    if (pencocokan_titik[1] == -1) {
      jumlah_kalimat <- 0
    } else {
      jumlah_kalimat <- length(pencocokan_titik)
    }
  }

  # Menampilkan hasil output
  cat(paste("Teks tersebut memuat", jumlah_kalimat, "kalimat dan",
    jumlah_kata, "kata.\n"))
}
```

OUTPUT SKENARIO UJI

```
# Program Menghitung Jumlah Kata dan Jumlah Kalimat

teks = input("Masukkan teks: ")

if teks.strip() == "":
  jumlah_kata = 0
  jumlah_kalimat = 0
else:
  jumlah_kata = len(teks.split())
  jumlah_kalimat = len(teks.split("."))-1

print("\nTeks tersebut memuat", jumlah_kalimat,
      "kalimat dan", jumlah_kata, "kata.")

... Masukkan teks: Media sosial atau disebut juga dengan jejaring sosial, seperti Facebook, Twitter, Instagram, dan masih ba

Teks tersebut memuat 5 kalimat dan 92 kata.
```

```
# Program Menghitung Jumlah Kata dan Jumlah Kalimat

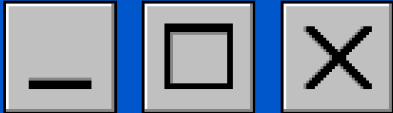
teks = input("Masukkan teks: ")

if teks.strip() == "":
  jumlah_kata = 0
  jumlah_kalimat = 0
else:
  jumlah_kata = len(teks.split())
  jumlah_kalimat = len(teks.split("."))-1

print("\nTeks tersebut memuat", jumlah_kalimat,
      "kalimat dan", jumlah_kata, "kata.")

... Masukkan teks:

Teks tersebut memuat 0 kalimat dan 0 kata.
```



DESKRIPSI SOAL

Program 2 berfokus pada penanganan string yang memuat karakter khusus seperti tanda petik tunggal dan ganda. Tugasnya adalah membuat empat variabel string (K1,K2,K3,K4) di Python dan R yang saat di *print* menghasilkan *output* persis sesuai soal. Tantangan utamanya adalah penulisan sintaks yang tepat agar karakter khusus tersebut tidak menyebabkan error saat program dijalankan.

KOMPONEN	KETERANGAN
INPUT	Membuat input objek string K1, K2, K3, dan K4 sesuai ketentuan soal
PROSES	Tidak ada proses
OUTPUT	Print isi K1, K2, K3, dan K4

BATASAN MASALAH

- Kalimat yang disimpan dalam K1, K2, K3, K4 harus sama persis dengan yang diminta soal.
- Program harus memastikan seluruh string ditulis dengan pasangan tanda petik yang lengkap agar tidak terjadi kesalahan sintaks (Syntax Error), Karakter khusus seperti garis miring (/), titik (.), dan tanda kutip harus tetap ditampilkan sesuai isi teks asli.
- Program hanya berfungsi untuk menyimpan dan menampilkan string, tidak ada perhitungan angka.

KONDISI KHUSUS

- 1.Kesalahan dalam penulisan tanda kutip dapat membuat program tidak bisa dijalankan.
- 2.Beberapa kalimat memerlukan karakter escape (\") agar dapat disimpan sebagai string dengan benar.
- 3.Hasil program harus sama persis dengan yang ada pada soal.

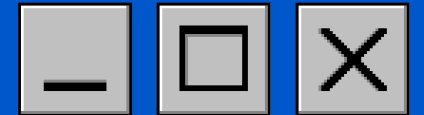
SOURCE CODE PYTHON

```
K1 = "Saya tak 'kan menyerah."  
K2 = 'Ia berkata, "Aku menyayangimu."'   
K3 = "\"Coba jelaskan pengertian 'cross-validation' dalam  
Machine Learning!\""  
K4 = "Surat keputusan itu bernomor 62/UN.34/19/2023."  
  
print(K1)  
print(K2)  
print(K3)  
print(K4)
```

SOURCE CODE R

```
K1 <- "Saya tak 'kan menyerah."  
K2 <- 'Ia berkata, "Aku menyayangimu."'   
K3 <- "\"Coba jelaskan pengertian 'cross-validation' dalam  
Machine Learning!\""  
K4 <- "Surat keputusan itu bernomor 62/UN.34/19/2023."  
  
cat(K1, "\n")  
## Saya tak 'kan menyerah.  
cat(K2, "\n")  
## Ia berkata, "Aku menyayangimu."  
cat(K3, "\n")  
## "Coba jelaskan pengertian 'cross-validation' dalam  
Machine Learning!"  
cat(K4, "\n")  
## Surat keputusan itu bernomor 62/UN.34/19/2023.
```


Program 3



DESKRIPSI SOAL

program dibuat untuk menentukan akar-akar persamaan kuadrat dengan bentuk umum :

$$ax^2 + bx + c = 0$$

dengan menggunakan rumus :

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Program harus mampu:

- Membaca nilai koefisien a,b,dan c
- Menghitung diskriminan
- Menentukan jenis akar:
 - Dua akar real berbeda
 - Akar real kembar
 - Akar imajiner
- Menampilkan hasil hingga tiga angka desimal

INPUT

variabel	Tipe data	Keterangan
a	float	koefisien (x^2)
b	float	koefisien (x)
c	float	konstanta

PROSES

1. Membaca nilai a, b, dan c
2. Menghitung diskriminan:

$$D = b^2 - 4ac$$

3. Melakukan pengecekan kondisi:
 - Jika $D > 0$ - dua akar real berbeda
 - Jika $D = 0$ - akar real kembar
 - Jika $D < 0$ - akar imajiner
4. Menghitung akar menggunakan rumus kuadrat

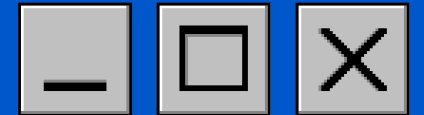
OUTPUT

- Nilai akar-akar persamaan kuadrat atau pesan bahwa akar bersifat imajiner.

SKENARIO UJI PROGRAM 3 – MENENTUKAN AKAR PERSAMAAN KUADRAT

No	Skenario	Input (a, b, c)	Diskriminan	Output yang	Status
1	Dua akar	(1, -5, 6)	1	$x_1 = 3.000$	Berhasil
2	Akar kembar	(1, -4, 4)	0	Akar kembar	Berhasil
3	Akar imajiner	(1, 2, 5)	-16	Persamaan	Berhasil
4	Koefisien	(2, -7, 3)	25	$x_1 = 3.000$	Berhasil
5	Akar negatif	(1, 6, 9)	0	Akar kembar	Berhasil
6	Nilai b = 0	(1, 0, -9)	36	$x_1 = 3.000$	Berhasil
7	Semua	(1, 3, 2)	1	$x_1 = -1.000$	Berhasil

Program 3



SOURCE CODE PYTHON

```
import math
def akar_kuadrat(a, b, c):
    D = b**2 - 4*a*c
    if D > 0:
        x1 = (-b + math.sqrt(D)) / (2*a)
        x2 = (-b - math.sqrt(D)) / (2*a)
        print(f"Akar pertama = {x1:.3f}")
        print(f"Akar kedua = {x2:.3f}")
    elif D == 0:
        x = -b / (2*a)
        print(f"Akar kembar = {x:.3f}")
    else:
        print("Persamaan memiliki akar-akar imajiner")

a = float(input("Masukkan a : "))
b = float(input("Masukkan b : "))
c = float(input("Masukkan c : "))

akar_kuadrat(a,b,c)
```

SOURCE CODE R

```
#AKAR REAL BERBEDA
# DATA CONTOH
A <- 1
B <- -5
C <- 6

D <- B^2 - 4*A*C

IF (D > 0) {

    X1 <- (-B + SQRT(D))/(2*A)
    X2 <- (-B - SQRT(D))/(2*A)

    CAT(sprintf("AKAR PERTAMA = %.3F\n", X1))
    CAT(sprintf("AKAR KEDUA = %.3F\n", X2))

} ELSE IF (D == 0) {

    X <- -B/(2*A)

    CAT(sprintf("AKAR KEMBAR = %.3F\n", X))

} ELSE {

    CAT("PERSAMAAN MEMILIKI AKAR-AKAR
    IMAJINER\n")

}
```

SKENARIO UJI

```
#Akar Real Berbeda
import math

def akar_kuadrat(a, b, c):

    D = b**2 - 4*a*c

    if D > 0:
        x1 = (-b + math.sqrt(D)) / (2*a)
        x2 = (-b - math.sqrt(D)) / (2*a)

        print(f"Akar pertama = {x1:.3f}")
        print(f"Akar kedua = {x2:.3f}")

    elif D == 0:
        x = -b / (2*a)

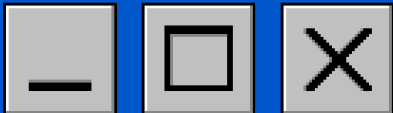
        print(f"Akar kembar = {x:.3f}")

    else:
        print("Persamaan memiliki akar-akar imajiner")

a = float(input("Masukkan a : "))
b = float(input("Masukkan b : "))
c = float(input("Masukkan c : "))

akar_kuadrat(a,b,c)
```

```
... Masukkan a : 1
Masukkan b : -5
Masukkan c : 6
Akar pertama = 3.000
Akar kedua = 2.000
```



DESKRIPSI SOAL

Program dibuat untuk mengekstrak dan menampilkan informasi tanggal lahir ASN berdasarkan 18 digit NIP yang diinputkan. Menggunakan teknik pemotongan string (string slicing), program mengambil 8 digit pertama NIP untuk memisahkan komponen tahun (4 digit), bulan (2 digit), dan tanggal (2 digit). Selanjutnya, struktur percabangan digunakan untuk mengonversi angka bulan menjadi nama bulan yang sesuai (misalnya, '01' menjadi 'Januari') sebelum menampilkan hasil akhirnya dalam format yang rapi.

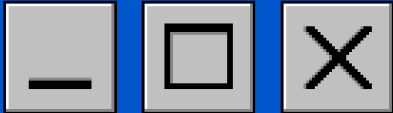
KOMPONEN	KETERANGAN
INPUT	String NIP ASN (contoh: 199301212019031010).
PROSES	1. Memotong (slicing) string NIP digit 1–4 untuk Tahun Lahir. 2. Memotong string digit 5–6 untuk Bulan Lahir. 3. Memotong string digit 7–8 untuk Tanggal Lahir. 4. Melakukan percabangan (If-Else) untuk mengonversi angka bulan (01-12) menjadi nama bulan ("Januari" - "Desember").
OUTPUT	Teks yang menampilkan Tanggal Lahir lengkap (Tanggal, Nama Bulan, Tahun).

Batasan Masalah

- Input wajib berupa string berisi 18 digit angka tanpa spasi atau karakter khusus.
- Program hanya memproses 8 digit pertama NIP untuk mengambil tahun (4 digit), bulan (2 digit), dan tanggal (2 digit).
- Nilai bulan yang diproses dibatasi pada rentang 01 sampai 12.
- Program hanya menampilkan informasi tanggal lahir (tanggal, nama bulan, tahun). Sisa 10 digit NIP lainnya (informasi pengangkatan, jenis kelamin, dan nomor urut) diabaikan oleh program.

Kondisi Khusus

- Penanganan jika panjang NIP kurang atau lebih dari 18 karakter.
- Penanganan jika pengguna memasukkan huruf, spasi, atau simbol.
- Kondisi jika digit bulan berada di luar rentang 01 sampai 12
- Kondisi jika digit tanggal berada di luar rentang 01 sampai 31 (atau tidak sesuai dengan jumlah hari pada bulan tersebut).

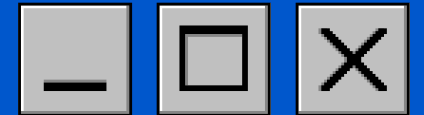


SOURCE CODE PYHTHON

```
def tampilkan_tanggal_lahir(nip):  
  
    if len(nip) < 8:  
        print("NIP tidak valid")  
    else:  
  
        tahun = nip[0:4]  
        bulan = nip[4:6]  
        tanggal = nip[6:8]  
  
        if bulan == "01":  
            nama_bulan = "Januari"  
        elif bulan == "02":  
            nama_bulan = "Februari"  
        elif bulan == "03":  
            nama_bulan = "Maret"  
        elif bulan == "04":  
            nama_bulan = "April"  
        elif bulan == "05":  
            nama_bulan = "Mei"  
        elif bulan == "06":  
            nama_bulan = "Juni"  
        elif bulan == "07":  
            nama_bulan = "Juli"  
        elif bulan == "08":  
            nama_bulan = "Agustus"  
        elif bulan == "09":  
            nama_bulan = "September"  
        elif bulan == "10":  
            nama_bulan = "Oktober"  
        elif bulan == "11":  
            nama_bulan = "November"  
        elif bulan == "12":  
            nama_bulan = "Desember"  
        else:  
            nama_bulan = "Bulan tidak valid"  
        print("Tanggal Lahir ASN:", tanggal, nama_bulan, tahun)  
  
# Memanggil fungsi  
tampilkan_tanggal_lahir("199301212019031010")
```

No	Jenis Skenario	Input NIP	Output Program
1	Normal (Valid)	199301212019031010	Tanggal Lahir: 21 Januari 1993
2	Norml (Valid)	200707052024012002	Tanggal Lahir: 07 Mei 2007
3	Kondisi Khusus (Digit Kurang)	20070705	Error: NIP harus berjumlah tepat 18 digit.
4	Kondisi Khusus (Tanggal Tidak Valid)	200705352019031010	Error: Format tanggal tidak valid (01-31).
5	Kondisi Khusus (Bulan Tidak Valid)	200713052019031010	Error: Format bulan tidak valid (01-12).

PROGRAM 4



SOURCE KODE R

```
tampilkan_tanggal_lahir <- function(nip) {
  if (nchar(nip) < 8) {
    cat("NIP tidak valid\n")
  } else {

    tahun<- substr(nip, 1, 4)
    bulan<- substr(nip, 5, 6)
    tanggal <- substr(nip, 7, 8)

    if (bulan == "01") {
      nama_bulan <- "Januari"
    } else if (bulan == "02") {
      nama_bulan <- "Februari"
    } else if (bulan == "03") {
      nama_bulan <- "Maret"
    } else if (bulan == "04") {
      nama_bulan <- "April"
    } else if (bulan == "05") {
      nama_bulan <- "Mei"
    } else if (bulan == "06") {
      nama_bulan <- "Juni"
    } else if (bulan == "07") {
      nama_bulan <- "Juli"
    } else if (bulan == "08") {
      nama_bulan <- "Agustus"
    } else if (bulan == "09") {
      nama_bulan <- "September"
    } else if (bulan == "10") {
      nama_bulan <- "Oktober"
    } else if (bulan == "11") {
      nama_bulan <- "November"
    } else if (bulan == "12") {
      nama_bulan <- "Desember"
    } else {
      nama_bulan <- "Bulan tidak valid"
    }
    cat("Tanggal Lahir ASN:", tanggal, nama_bulan, tahun, "\n")
  }
}

tampilkan_tanggal_lahir("199301212019031010")
```

OUTPUT SKENARIO UJI

```
[1]
✓ Od ▶ def tampilkan_tanggal_lahir(nip):

    if len(nip) < 8:
        print("NIP tidak valid")
    else:

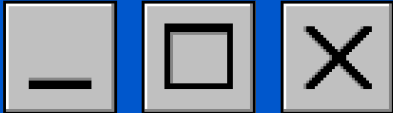
        tahun = nip[0:4]
        bulan = nip[4:6]
        tanggal = nip[6:8]

        if bulan == "01":
            nama_bulan = "Januari"
        elif bulan == "02":
            nama_bulan = "Februari"
        elif bulan == "03":
            nama_bulan = "Maret"
        elif bulan == "04":
            nama_bulan = "April"
        elif bulan == "05":
            nama_bulan = "Mei"
        elif bulan == "06":
            nama_bulan = "Juni"
```

```
[1]
✓ Od ▶ elif bulan == "07":
        nama_bulan = "Juli"
    elif bulan == "08":
        nama_bulan = "Agustus"
    elif bulan == "09":
        nama_bulan = "September"
    elif bulan == "10":
        nama_bulan = "Oktober"
    elif bulan == "11":
        nama_bulan = "November"
    elif bulan == "12":
        nama_bulan = "Desember"
    else:
        nama_bulan = "Bulan tidak valid"
    print("Tanggal Lahir ASN:", tanggal, nama_bulan, tahun)

# Memanggil fungsi
tampilkan_tanggal_lahir("199301212019031010")

... Tanggal Lahir ASN: 21 Januari 1993
```



DESKRIPSI SOAL

Program digunakan untuk menentukan cluster terdekat dari suatu titik $U = (x_1, x_2, x_3)$ berdasarkan jarak Euclidean terhadap tiga pusat cluster yang telah ditentukan, yaitu:

Cluster A = (2, 1, 3)
Cluster B = (1, -4, 6)
Cluster C = (-2, 3, -2)

Program menghitung jarak titik U ke masing-masing cluster, kemudian menentukan cluster yang memiliki jarak paling kecil.

Rumus untuk menghitung jarak:

$$jarak = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + (x_3 - y_3)^2}$$

KOMPONEN	KETERANGAN
INPUT	x1, x2, x3 (koordinat titik U)
PROSES	Menghitung jarak Euclidean titik U ke Cluster A, B, dan C, lalu membandingkan hasilnya
OUTPUT	Nama cluster terdekat (A, B, atau C) serta nilai jaraknya

Batas masalah:

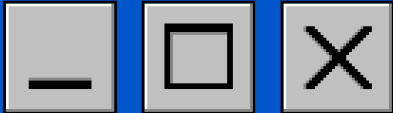
- Program menggunakan tiga pusat cluster yang nilainya tetap:
 - A = (2, 1, 3)
 - B = (1, -4, 6)
 - C = (-2, 3, -2)
- Nilai x_1 , x_2 , dan x_3 berupa bilangan numerik (integer atau desimal).
- Perhitungan jarak menggunakan rumus Euclidean 3 dimensi
- program hanya menentukan cluster berdasarkan jarak terkecil.

kondisi khusus yang harus ditangani oleh program:

Kondisi Normal

- Titik U memiliki satu jarak terkecil yang jelas terhadap salah satu cluster.
- Program mengembalikan Cluster A, B, atau C.
- Titik U berada sangat dekat atau tepat pada pusat suatu cluster.

No	Jenis Skenario	Input	Output Program
1	NORMAL	(7,-2,4)	Cluster A
2	NORMAL	(2,-5,7)	cluster B
3	KHUSUS	(-1,3,-2)	Cluster C



SOURCE CODE PYTHON

```
import math

# Pusat cluster
A = (2, 1, 3)
B = (1, -4, 6)
C = (-2, 3, -2)

# Titik U
U = (1, 2, 3)

# Menghitung jarak ke masing-masing cluster
jarak_A = math.sqrt((U[0] - A[0])**2 + (U[1] - A[1])**2 + (U[2] - A[2])**2)
jarak_B = math.sqrt((U[0] - B[0])**2 + (U[1] - B[1])**2 + (U[2] - B[2])**2)
jarak_C = math.sqrt((U[0] - C[0])**2 + (U[1] - C[1])**2 + (U[2] - C[2])**2)

print("Jarak ke A =", jarak_A)
print("Jarak ke B =", jarak_B)
print("Jarak ke C =", jarak_C)

# Menentukan cluster terdekat
if jarak_A < jarak_B and jarak_A < jarak_C:
    print("Titik U termasuk Cluster A")
elif jarak_B < jarak_A and jarak_B < jarak_C:
    print("Titik U termasuk Cluster B")
else:
    print("Titik U termasuk Cluster C")
```

SOURCE CODE R

```
# Titik pusat cluster
A <- c(2, 1, 3)
B <- c(1, -4, 6)
C <- c(-2, 3, -2)

# Input data
x1 <- 1
x2 <- 2
x3 <- 3

U <- c(x1, x2, x3)

# Jarak ke cluster A
jarakA <- sqrt((U[1]-A[1])^2 +
(U[2]-A[2])^2 +
(U[3]-A[3])^2)

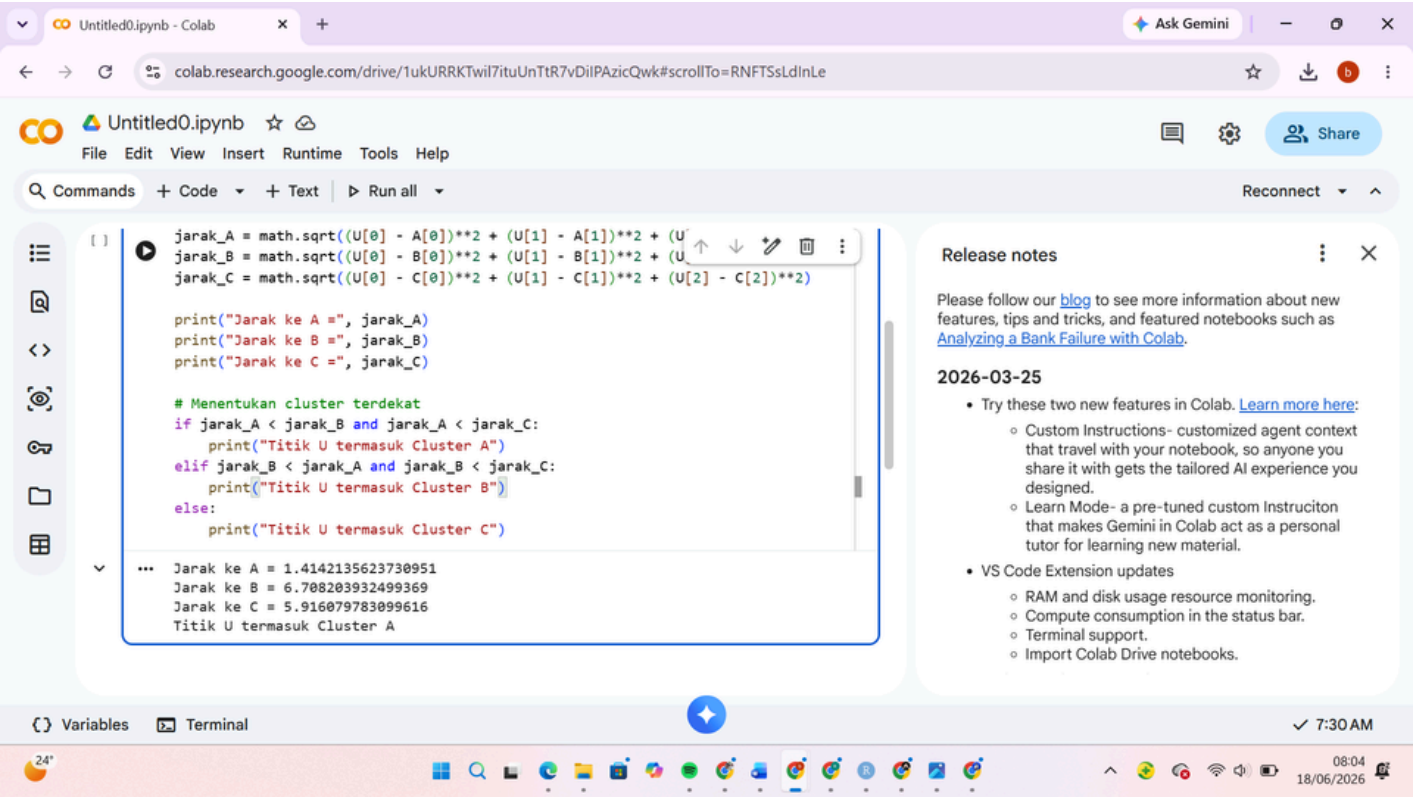
# Jarak ke cluster B
jarakB <- sqrt((U[1]-B[1])^2 +
(U[2]-B[2])^2 +
(U[3]-B[3])^2)

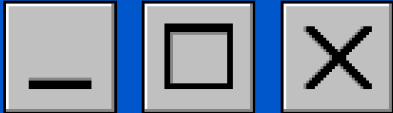
# Jarak ke cluster C
jarakC <- sqrt((U[1]-C[1])^2 +
(U[2]-C[2])^2 +
(U[3]-C[3])^2)

# Menentukan cluster
if (jarakA <= jarakB && jarakA <= jarakC) {
  cluster <- "A"
} else if (jarakB <= jarakA && jarakB <= jarakC) {
  cluster <- "B"
} else {
  cluster <- "C"
}

cat("Jarak ke A =", jarakA, "\n")
## Jarak ke A = 1.414214
cat("Jarak ke B =", jarakB, "\n")
## Jarak ke B = 6.708204
cat("Jarak ke C =", jarakC, "\n")
## Jarak ke C = 5.91608
cat("Cluster =", cluster)
## Cluster = A
```

OUTPUT SKENARIO UJI





Analisis Permasalahan

Dalam statistika, interval kepercayaan digunakan untuk memperkirakan nilai parameter populasi berdasarkan data sampel. Pada program ini parameter yang dihitung adalah proporsi populasi. Karena tidak memungkinkan meneliti seluruh populasi, maka digunakan proporsi sampel ($\hat{p}$) untuk memperkirakan proporsi populasi sebenarnya. Hasil estimasi ditampilkan dalam bentuk interval kepercayaan yang terdiri dari batas bawah dan batas atas. Rumus interval kepercayaan proporsi populasi:

Pseudocode

- 1. Start
- 2. Input nilai $\hat{p}$
- 3. Input ukuran sampel n
- 4. Input nilai α
- 5. Jika $\hat{p} < 0$ atau $\hat{p} > 1$:
Tampilkan pesan kesalahan
Jika tidak:
Jika $\alpha = 0.10$ maka $z = 1.645$
Jika $\alpha = 0.05$ maka $z = 1.96$
Hitung margin error
Hitung batas bawah
Hitung batas atas
Tampilkan interval kepercayaan
- 6. End

Output Skenario Uji

```
import math

# Input data
p = float(input("Masukkan proporsi sampel (p̂): "))
n = int(input("Masukkan ukuran sampel (n): "))
alpha = float(input("Masukkan nilai alpha (0.10 atau 0.05): "))

# Validasi nilai proporsi
if p < 0 or p > 1:
    print("Error: Nilai proporsi harus antara 0 dan 1")
else:
    # Menentukan nilai z berdasarkan alpha
    if alpha == 0.10:
        z = 1.645
    elif alpha == 0.05:
        z = 1.96
    else:
        print("Error: Alpha harus 0.10 atau 0.05")
        exit()

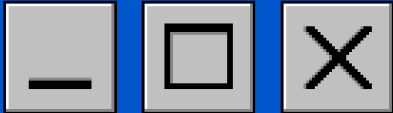
    # Menghitung margin of error
    margin_error = z * math.sqrt((p * (1 - p)) / n)

    # Menghitung batas bawah dan batas atas
    batas_bawah = p - margin_error
    batas_atas = p + margin_error

    # Menampilkan hasil
    print("\nInterval Kepercayaan Proporsi Populasi")
    print("-----")
    print("Batas bawah =", round(batas_bawah, 4))
    print("Batas atas  =", round(batas_atas, 4))

... Masukkan proporsi sampel (p̂): 0.6
Masukkan ukuran sampel (n): 100
Masukkan nilai alpha (0.10 atau 0.05): 0.05

Interval Kepercayaan Proporsi Populasi
-----
Batas bawah = 0.504
Batas atas  = 0.696
```

SOURCE CODE PYTHON

```
import math
# Input data
p = float(input("Masukkan proporsi sampel (p): "))
n = int(input("Masukkan ukuran sampel (n): "))
alpha = float(input("Masukkan nilai alpha (0.10 atau 0.05): "))
# Validasi nilai proporsi
if p < 0 or p > 1:
    print("Error: Nilai proporsi harus antara 0 dan 1")
else:
    # Menentukan nilai z
    if alpha == 0.10:
        z = 1.645
    elif alpha == 0.05:
        z = 1.96
    else:
        print("Error: Alpha harus 0.10 atau 0.05")
        exit()
    # Menghitung margin of error
    margin_error = z * math.sqrt((p * (1 - p)) / n)
    # Menghitung interval kepercayaan
    batas_bawah = p - margin_error
    batas_atas = p + margin_error
    # Menampilkan hasil
    print("\nInterval Kepercayaan Proporsi Populasi")
    print("-----")
    print("Batas bawah =", round(batas_bawah, 4))
    print("Batas atas =", round(batas_atas, 4))
```

SOURCE CODE R

```
# Input data
p <- 0.6
n <- 100
alpha <- 0.05
# Menentukan nilai z
if (alpha == 0.05) {
    z <- 1.96
} else if (alpha == 0.10) {
    z <- 1.645
} else {
    print("Alpha tidak valid")
}
# Menghitung margin of error
margin_error <- z * sqrt((p * (1 - p)) / n)
# Menghitung interval kepercayaan
batas_bawah <- p - margin_error
batas_atas <- p + margin_error
# Menampilkan hasil
cat("Interval Kepercayaan Proporsi Populasi\n")
cat("-----\n")
cat("Batas bawah =", round(batas_bawah, 4), "\n")
cat("Batas atas =", round(batas_atas, 4), "\n")
```

Tabel Pengujian

No	Input	Alpha	Margin of Error	Batas Bawah	Batas Atas	Keterangan
1	p = 0.6, n = 100	0.05	0.096	0.504	0.696	Valid
2	p = 0.4, n = 50	0.1	0.1142	0.2858	0.5142	Valid
3	p = 1.5, n = 80	0.05	-	-	-	Tidak valid

closing



THANK YOU